



CSS Fundamentals

Styling the Web with Cascading Style Sheets

Chapter Overview

- 01 Introduction to CSS
- 02 CSS Selectors & Application
- 03 Styling Text with CSS
- 04 The Box Model
- 05 Layout & Display
- 06 Responsive Design & CSS Variables



Introduction to CSS

Section 1



What is CSS?

Definition

- Cascading Style Sheets — the language that styles HTML documents
- Controls colors, fonts, spacing, layout, and visual effects
- Separates presentation from content — HTML handles structure, CSS handles appearance

Why CSS Matters

- One stylesheet can control the look of an entire website
- Change the design without touching the HTML
- Enables responsive design — pages adapt to different screen sizes



The Cascade

What Does "Cascading" Mean?

- Multiple style rules can apply to the same element
- The cascade algorithm decides which rule wins based on three factors

The Three Cascade Factors

- Origin — where the style comes from (browser defaults → author styles → !important)
- Specificity — how targeted the selector is (element < class < ID < inline)
- Source order — when specificity is equal, the last rule defined wins



Cascade Priority Hierarchy

Priority (lowest to highest)

- User-agent styles — browser defaults (e.g., blue underlined links)
- Author styles — your external, internal, or embedded CSS
- Specificity — more specific selectors override less specific ones
- Inline styles — `style="..."` on an element overrides stylesheet rules
- `!important` — overrides everything (use sparingly!)

Specificity Scoring

- Element selectors (`p`, `div`) = 0-0-1 | Class selectors (`.card`) = 0-1-0
- ID selectors (`#header`) = 1-0-0 | Inline styles = highest

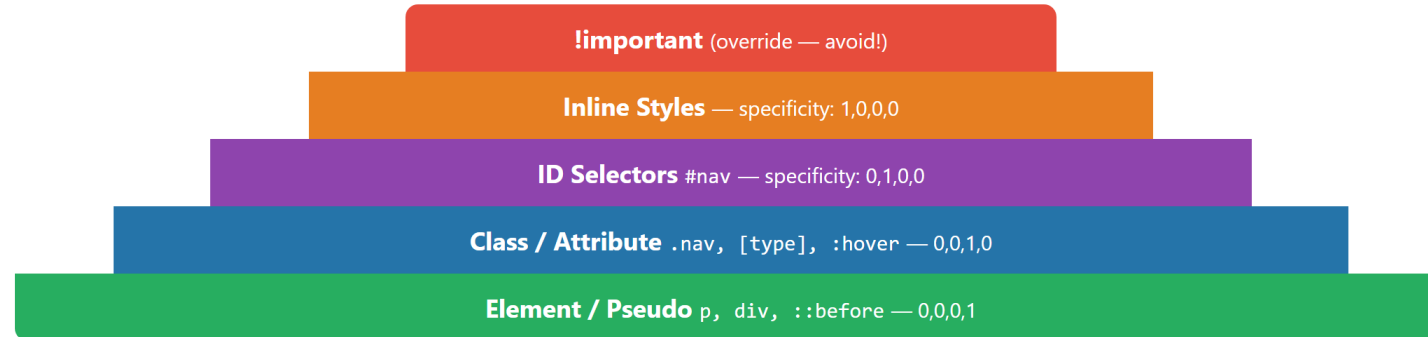


Specificity in Action

```
/* Specificity: 0-0-1 (element selector) */  
p { color: black; }  
  
/* Specificity: 0-1-0 (class selector – wins!) */  
.intro { color: blue; }  
  
/* Specificity: 1-0-0 (ID selector – wins over class) */  
#welcome { color: green; }  
  
/* Inline style overrides all of the above */  
/* <p id="welcome" class="intro" style="color: red;"> */  
  
/* !important overrides everything – avoid when possible */  
p { color: purple !important; }
```

CSS Specificity Hierarchy

CSS Specificity & Cascade



Which Rule Wins?

```
p { color: blue; }           /* 0,0,0,1 */
.intro { color: red; }       /* 0,0,1,0 */
#main { color: green; }      /* 0,1,0,0 */
style="color:pink"          /* 1,0,0,0 */
```

- Lowest priority
- Beats element
- Beats class
- **Wins!** (highest)

CSS Syntax

Rule Structure

- A CSS rule has a selector and a declaration block
- Selector targets which HTML elements to style
- Declaration block uses curly braces { } with property: value pairs

Declarations

- Each declaration has a property and a value separated by a colon
- Declarations end with a semicolon



CSS Syntax Example

```
/* Selector targets all <h1> elements */
h1 {
  color: #7500C0;           /* text color */
  font-size: 24px;          /* font size in pixels */
  text-align: center;       /* center the text */
  margin-bottom: 16px;      /* space below */
}

/* Multiple declarations in one rule */
p {
  color: #333333;
  font-family: Arial, sans-serif;
  line-height: 1.6;
}
```

CSS Selectors & Application

Section 2



CSS Selector Types

CSS Selector Types

Element Selector

```
p { color: blue; }
```

Selects ALL <p> elements

Class Selector

```
.highlight { color: red; }
```

Selects class="highlight"

ID Selector

```
#header { font-size: 20px; }
```

Selects id="header" (unique)

Universal Selector

```
* { margin: 0; }
```

Selects ALL elements

Grouping & Combinators

```
h1, h2, h3 { color: green; }
```

Group: same styles to multiple

```
div p { }    div > p { }
```

Descendant & direct child

Element, ID, and Class Selectors

Element Selector

- Targets all elements of a type: `p { color: blue; }`
- Applies to every matching element on the page

ID Selector

- Targets one unique element: `#header { background: gray; }`
- IDs must be unique — only one element per ID on a page

Class Selector

- Targets elements with a class: `.highlight { color: red; }`
- Multiple elements can share the same class



Universal and Grouping Selectors

Universal Selector

- `* { margin: 0; padding: 0; }` — targets every element
- Often used for CSS resets to remove default browser styling

Grouping Selector

- Apply the same styles to multiple selectors with commas
- `h1, h2, h3 { font-family: Georgia; }` — styles all three headings
- Reduces repetition and keeps stylesheets shorter



Selector Examples

```
/* Element selector – all paragraphs */  
p { color: #333; }  
  
/* ID selector – one unique element */  
#main-title { font-size: 32px; }  
  
/* Class selector – reusable */  
.warning { color: red; font-weight: bold; }  
  
/* Universal selector – CSS reset */  
* { margin: 0; padding: 0; box-sizing: border-box; }  
  
/* Grouping selector – shared styles */  
h1, h2, h3 { font-family: 'Poppins', sans-serif; }
```

How to Add CSS to HTML

Inline CSS

- Applied directly on an element: `<p style="color: blue;">`
- Highest specificity but hardest to maintain

Internal (Embedded) CSS

- Written inside `<style>` tags in the `<head>` section
- Good for single-page styling or quick prototyping

External CSS

- Separate .css file linked with `<link>` tag
- Best practice — one file styles the entire site



Linking an External Stylesheet

```
<!-- In your HTML <head> section -->
<head>
  <meta charset="UTF-8">
  <title>My Page</title>
  <link rel="stylesheet" href="styles.css">
</head>

/* styles.css – separate file */
body {
  font-family: Arial, sans-serif;
  background-color: #f5f5f5;
}

h1 { color: #7500C0; }
.container { max-width: 960px; margin: 0 auto; }
```



CSS Application Best Practices

Prefer External Stylesheets

- Keeps HTML clean and separates concerns
- Browser caches the CSS file — faster page loads on repeat visits

Avoid Inline Styles

- Difficult to maintain and override
- Cannot reuse styles across elements
- Use meaningful class names that describe purpose, not appearance
 - `.error-message` is better than `.red-text`



Styling Text with CSS

Section 3



Text & Font Properties Overview

CSS Text & Font Properties

Text Properties

color	#333, red, rgb()
text-align	left center right justify
text-decoration	none underline line-through
text-transform	uppercase lowercase capitalize
line-height	1.5, 24px, 150%

Font Properties

font-family	Arial, 'Helvetica', sans-serif
font-size	16px, 1.2em, 1rem
font-weight	normal bold 100-900
font-style	normal italic oblique

Color Formats

Named: red, blue

Hex: #E74C3C

RGB: rgb(231, 76, 60)

RGBA: rgba(0,0,0,0.5)

HSL: hsl(6, 78%, 57%)

Use rem for font-size, em for spacing, and px for borders



Text Color and Background

Text Color

- color property sets the text color
- Values: named colors (red), hex (#FF0000), rgb(255,0,0), hsl(0,100%,50%)

Background Color

- background-color sets the element's background
- Can use any color format: hex, rgb, hsl, or named colors
- Contrast matters — ensure text is readable against its background

Text Alignment and Decoration

Text Alignment

- `text-align: left | right | center | justify`
- Controls horizontal alignment of text within its container

Text Decoration

- `text-decoration: underline | overline | line-through | none`
- Often used to remove underlines from links: `a { text-decoration: none; }`

Text Transformation

- `text-transform: uppercase | lowercase | capitalize`
- Changes text appearance without modifying the HTML content



Text Spacing Properties

Text Indentation

- `text-indent: 40px` — indents the first line of a paragraph

Letter Spacing

- `letter-spacing: 2px` — adjusts space between characters
- Useful for headings and uppercase text

Line Height

- `line-height: 1.6` — sets the vertical spacing between lines
- Unitless values (1.5, 1.6) are preferred over fixed pixel values

Word Spacing

- `word-spacing: 4px` — adjusts space between words



Text Styling Examples

```
/* Color and background */
h1 { color: #7500C0; background-color: #F3E8FC; }

/* Alignment and decoration */
.centered { text-align: center; }
a { text-decoration: none; color: #2574A9; }
a:hover { text-decoration: underline; }

/* Transformation and spacing */
.uppercase { text-transform: uppercase; letter-spacing: 2px; }
p { line-height: 1.6; text-indent: 20px; }

/* Removing default link underline */
.nav-link { text-decoration: none; font-weight: bold; }
```


CSS Fonts and Font Families

Generic Font Families

- serif — letters have small decorative strokes (Times New Roman, Georgia)
- sans-serif — clean, no strokes (Arial, Helvetica, Poppins)
- monospace — equal character width (Consolas, Courier New)

The font-family Property

- Specify a font stack — browser uses the first available font
- Always end with a generic family as fallback



Font Weight, Style, and Size

Font Weight

- font-weight: normal | bold | 100-900
- 400 = normal, 700 = bold

Font Style

- font-style: normal | italic | oblique

Font Size

- px — fixed pixel size (16px), predictable but not scalable
- em — relative to parent element's font size (1.5em = 150%)
- rem — relative to root element's font size (consistent across nesting)



Web Fonts and @font-face

Web Fonts

- Not limited to fonts installed on the user's computer
- Google Fonts — free library of 1600+ web font families
- Add via <link> tag in HTML or @import in CSS

@font-face Rule

- Load custom font files hosted on your own server
- Define font-family name and src URL to the font file
- Supports .woff2 (best compression), .woff, .ttf formats



Font Examples

```
/* Font stack with fallbacks */
body { font-family: 'Poppins', Arial, sans-serif; }
code { font-family: 'Consolas', 'Courier New', monospace; }

/* Google Fonts (add <link> in HTML first) */
h1 { font-family: 'Roboto', sans-serif; font-weight: 700; }

/* Font size: px vs em vs rem */
html { font-size: 16px; }           /* base size */
h1    { font-size: 2rem; }          /* 32px */
h2    { font-size: 1.5rem; }        /* 24px */
small { font-size: 0.875em; }       /* 14px */

/* @font-face – self-hosted font */
@font-face {
  font-family: 'MyFont';
  src: url('fonts/myfont.woff2') format('woff2');
}
```

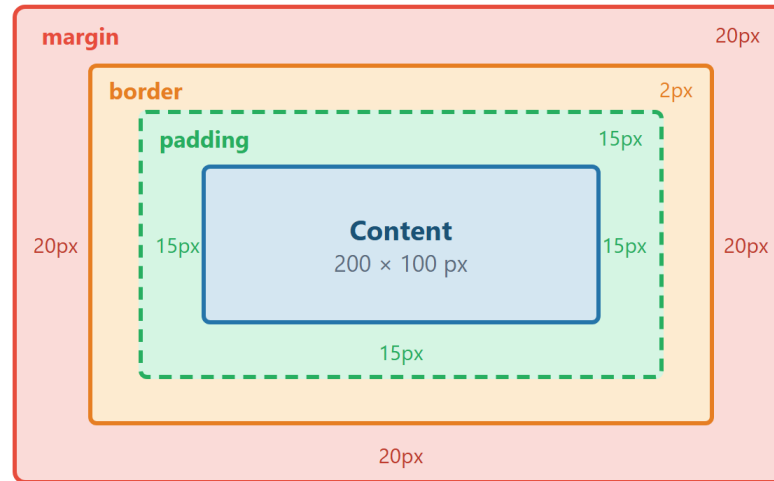
The Box Model

Section 4



The CSS Box Model

The CSS Box Model



Content
width: 200px; height: 100px;

Padding
padding: 15px;

Border
border: 2px solid black;

Margin
margin: 20px;

Total width = content + padding + border + margin
box-sizing: border-box;
^ includes padding + border in width

Box Model Components

Content

- The actual text, image, or other content of the element
- Sized with width and height properties

Padding

- Space between the content and the border — inside the element
- Background color extends through the padding area

Border

- Surrounds the padding — visible line around the element

Margin

- Space outside the border — separates elements from each other



Width, Height, and Box Sizing

content-box (Default Behavior)

- width and height set the content area only
- Total size = width + padding + border (wider than you set!)

border-box (Recommended)

- width and height include padding and border
- Element stays the exact width you set — much easier to work with
- Most developers apply it globally with a CSS reset

Max-Width

- max-width: 960px — element won't grow beyond this but can shrink



content-box vs border-box

```
/* content-box (default) – total width = 300 + 40 + 2 = 342px */
.content-box {
  box-sizing: content-box;
  width: 300px;
  padding: 20px;
  border: 1px solid #333;
}

/* border-box – total width stays 300px (content shrinks) */
.border-box {
  box-sizing: border-box;
  width: 300px;
  padding: 20px;
  border: 1px solid #333;
}

/* Recommended: apply border-box globally */
*, *::before, *::after { box-sizing: border-box; }
```

Margins and Padding

Shorthand Syntax

- `margin: 10px` — all four sides
- `margin: 10px 20px` — top/bottom, left/right
- `margin: 10px 20px 30px 40px` — top, right, bottom, left (clockwise)

Individual Sides

- `margin-top`, `margin-right`, `margin-bottom`, `margin-left`
- Same pattern for `padding-top`, `padding-right`, etc.

Auto Margins

- `margin: 0 auto` — horizontally centers a block element



Borders, Outline, and Border-Radius

Border Property

- border: 1px solid #333 — width, style, color in one declaration
- Styles: solid, dashed, dotted, double, none
- Can set individual sides: border-bottom: 2px solid #7500C0

Outline

- Drawn outside the border — does not affect layout or box size
- Used for focus indicators on form elements

Border-Radius

- Rounds the corners: border-radius: 8px
- 50% creates a circle (on square elements)



Box Model in Practice

```
/* Common box-sizing reset */
*, *::before, *::after {
  box-sizing: border-box;
}

/* Card component */
.card {
  width: 300px;
  max-width: 100%;
  padding: 20px;
  margin: 16px auto;          /* centered */
  border: 1px solid #ddd;
  border-radius: 8px;
  background-color: #fff;
}

/* Outline on focus (accessibility) */
input:focus { outline: 2px solid #7500C0; }
```

Layout & Display

Section 5



Display Property Comparison

CSS Display Types

display: block

div — full width

p — stacks vertically

h1 — line break before/after

- ✓ Width & height apply
- ✓ Takes full container width
- ✓ Stacks vertically

display: inline

Text with `span` and `a` and

`strong` inline.

- ✗ Width & height ignored
- ✓ Stays in text flow
- ✓ Only as wide as content

display: inline-block

Box 1

Box 2

Box 3

- ✓ Width & height apply
- ✓ Sits inline with others
- ✓ Best of both worlds

Use `display: none` to hide elements completely — `visibility: hidden` hides but keeps space



The Display Property

Block Elements

- Take up the full available width — stack vertically
- Default for <div>, <p>, <h1>–<h6>, <section>, <header>

Inline Elements

- Only take up as much width as their content — flow horizontally
- Default for , <a>, , ,
- Width and height are ignored — but padding and horizontal margins still apply
- Vertical padding/margins render visually but don't push other elements

Inline-Block

- Flows inline like text but accepts width, height, and all margins/padding



Hiding Content and Overflow

Hiding Elements

- `display: none` — removes element from layout entirely
- `visibility: hidden` — hides element but keeps its space in the layout

CSS Overflow

- Controls what happens when content is too large for its container
- `overflow: visible` (default) — content spills out
- `overflow: hidden` — clips content at the container boundary
- `overflow: scroll` — always shows scrollbars
- `overflow: auto` — shows scrollbars only when needed



Floating Elements

The float Property

- float: left | right — element floats to one side, text wraps around it
- Originally designed for wrapping text around images

Clearing Floats

- clear: both — forces the next element below floated elements
- The clearfix hack: parent uses overflow: auto to contain floated children

Modern Alternatives

- Flexbox and CSS Grid are preferred for page layouts (covered in a dedicated module)
- Floats are still useful for wrapping text around images



Layout Examples

```
/* Block vs inline vs inline-block */
.block { display: block; width: 100%; }
.inline { display: inline; }
.ib-btn { display: inline-block; padding: 8px 16px; }

/* Hide elements */
.hidden { display: none; }           /* removed from flow */
.invisible { visibility: hidden; }  /* space preserved */

/* Overflow */
.scroll-box {
  height: 200px;
  overflow: auto;           /* scroll if needed */
}

/* Float an image */
img.left { float: left; margin-right: 16px; }
```

Responsive Design & CSS Variables

Section 6



Media Queries

What Are Media Queries?

- Apply CSS rules only when certain conditions are met (screen size, orientation)
- Foundation of responsive web design

Mobile-First Approach

- Write base styles for small screens first
- Use min-width queries to add styles for larger screens

Common Breakpoints

- 768px — tablets | 1024px — small desktops | 1200px — large screens



Responsive Design with Media Queries

```
/* Mobile-first: base styles for small screens */
.container { padding: 10px; }
.card-row { display: flex; flex-direction: column; gap: 16px; }

/* Tablet and up (768px+) */
@media (min-width: 768px) {
  .container { padding: 20px; max-width: 720px; margin: 0 auto; }
  .card-row { flex-direction: row; flex-wrap: wrap; }
}

/* Desktop (1024px+) */
@media (min-width: 1024px) {
  .container { max-width: 960px; }
}
```

Responsive Units

Relative Length Units

- % — percentage of the parent element's size
- vw / vh — percentage of the viewport width / height
- rem — relative to root font size (consistent across nesting)

Modern Sizing Functions

- clamp(min, preferred, max) — fluid sizing with built-in limits
- min() / max() — pick the smaller or larger of two values



Responsive Units in Practice

```
/* Fluid typography – scales with viewport but stays in range */
h1 { font-size: clamp(1.5rem, 4vw, 3rem); }
p { font-size: clamp(1rem, 2vw, 1.25rem); }

/* Fluid container */
.container { width: min(90%, 1200px); margin: 0 auto; }

/* Full-viewport hero section */
.hero { height: 100vh; padding: 5vw; }

/* Responsive spacing with rem */
:root { font-size: 16px; }
section { padding: 2rem; } /* 32px, scales with root */
h2 { margin-bottom: 1rem; }
```

CSS Custom Properties (Variables)

What Are CSS Variables?

- Reusable values defined with `--name` syntax and accessed with `var()`
- Defined on `:root` for global access, or on any element for scoped use

Benefits

- Maintainability — change one variable, update everywhere it's used
- Consistency — keep colors, spacing, and fonts in one place
- Design systems — build a complete theme with variables
- Can be updated dynamically with JavaScript (unlike Sass variables)



CSS Variables in Action

```
/* Define variables on :root for global access */
:root {
  --primary-color: #7500C0;
  --text-color: #333333;
  --spacing: 16px;
  --border-radius: 8px;
  --font-main: 'Poppins', sans-serif;
}

/* Use variables anywhere */
h1 { color: var(--primary-color); font-family: var(--font-main); }
p { color: var(--text-color); margin-bottom: var(--spacing); }

.card {
  border: 1px solid var(--primary-color);
  border-radius: var(--border-radius);
  padding: var(--spacing);
}
```

Key Takeaways

Key Takeaways

1. CSS separates presentation from content — style HTML without changing structure
2. The cascade resolves conflicts via origin, specificity, and source order
3. Use box-sizing: border-box globally — it makes width/height predictable
4. Block, inline, and inline-block control how elements flow on the page
5. Media queries enable responsive design — mobile-first with min-width breakpoints
6. CSS variables (--name / var()) keep your styles maintainable and consistent

